Development Plan
FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

Table 1: Revision History

Date	**Developer(s)**	**Change**
Sep 27 2026	Ibrahim Tariq	Add Introduction and Gen AI usage in report sections
Sep 27 2026	Vikram Chandar	Add Expected Technology, Coding Standards and Use of AI sections
Sep 27 2026	Sankalpa Chhetri Dhakal	Add IP to Protect section and Project Decomposition and Scheduling sections
Sep 28 2026	John Cho	Added POC Demo Plan and Potential Risks
Sep 28 2026	Arjun Sasidaran	Add Copyright License, Team Meeting Plan, Team Communication Plan, Team Member Roles and Workflow Plan sections, and reflection
Oct 19 2026	Arjun Sasidaran	Added user accounts and database storage to the stretch goals in the scheduling plan

This Development Plan explains how the team will organize and manage the FPVR project. It covers how the team will communicate, hold meetings, divide responsibilities, track tasks, and manage the development process. It also describes the project schedule, proof of concept, expected technologies, use of AI tools, and coding standards. These plans will help the team stay organized, track progress, and work together throughout the project.

1 Confidential Information?

FPVR is not subject to any form of confidentiality nor should it contain any private interactions or insider information from stakeholders that might be treated as being confidential. Any potentially sensitive information that needs to be recorded will be stored separate from any public repository.

2 IP to Protect

There is currently no intellectual property agreement associated with this project. The project is being developed by the team as part of the capstone course and is not subject to any additional IP protection requirements from an external organization.

3 Copyright License

The team has adopted the MIT License for this project. The MIT License is a short, permissive open-source license that allows anyone to use, modify, and distribute the software, provided that the original copyright notice and license are included. The team chose this license because it is simple to understand and places few restrictions on others who may want to build on the project. The full license is available in the `LICENSE` file in the project repository.

4 Team Meeting Plan

The team holds regular internal meetings and also meets with the project supervisor, Dr. Douglas Down.

- **Team meetings:** The team meets once a week, in person, on Thursdays from 3:30 to 4:20 p.m. If the team is unable to meet in person, the meeting is held online on Discord at the same time.

- **Supervisor meetings:** The team meets with Dr. Down in person in his office every two weeks. The exact meeting time will be finalized once a time is confirmed with and approved by the supervisor.
- **Chair and agenda:** Every meeting has a chair and an agenda. The chair posts the agenda in the meeting's GitHub issue at least one day before the meeting.
- **Structure:** Each meeting follows the same structure:
  1. Review of action items from the last meeting.
  2. Progress updates from each member.
  3. Discussion of blockers and decisions.
  4. Assignment of new action items.
- **Notes:** Meeting notes and action items are recorded in a GitHub issue created from the matching meeting template (team meeting or supervisor meeting).

5 Team Communication Plan

The team uses a small set of tools for communication, each with a specific purpose, so that discussions are easy to find and decisions are recorded.

- **Discord:** used for day-to-day messaging, quick questions, and coordinating work. Online team meetings are also held on Discord.
- **GitHub Issues:** used for tasks, bugs, meeting agendas and notes, and decisions, so that there is a written record. Meetings are created from the team's issue templates, and any decision made on Discord is recorded in the relevant issue.
- **Pull request comments:** used for discussion and review of code and documentation changes.
- **External communication:** the team liaison (John Cho) is the main point of contact with the supervisor, TA, and instructor, and shares any updates with the team on Discord.
- **Response time:** members are expected to respond to messages within 24 hours on weekdays.
- **Urgent matters:** handled by mentioning the relevant members directly on Discord, followed by text message if there is no response.

6 Team Member Roles

Each team member has both project-level roles, which support how the team works together, and a technical role, which covers an area of the system they lead.

6.1 Project Roles

- **Team lead (Ibrahim Tariq):** tracks deadlines and keeps the project on schedule.
- **Team liaison (John Cho):** the main point of contact with the supervisor, TA, and instructor.
- **Meeting chair:** sets the agenda and runs the meeting. This role rotates between meetings.
- **Notetaker:** records meeting notes and action items in the meeting's GitHub issue. This role rotates between meetings.
- **Quality assurance:** checks each deliverable against its checklist and the Avenue rubric before submission. This role rotates for each deliverable, and the member who wrote the most of a deliverable does not act as its QA reviewer.
- **Reviewer:** reviews pull requests. Every member reviews, and each pull request needs approval from at least one member other than its author.

6.2 Technical Roles

Each member leads one technical area of the project, as shown in Table 2. The lead is responsible for the design of that area and is the first person to ask about it, but is not the only person who works on it.

Team Member	Technical Area
John Cho	Controller input and hardware integration
Ibrahim Tariq	Flight physics and VR / camera
Sankalpa Chhetri Dhakal	Training environments and ML / AI
Vikram Chandar	Back-end (game state, scoring, flight data)
Arjun Sasidaran	Back-end (game state, scoring, flight data)

Table 2: Technical area leads

6.3 Flexibility

Roles are assigned with overlap and are expected to change throughout the year. Every member contributes to both development and documentation, and work is reassigned based on workload and deadlines rather than strictly by role.

7 Workflow Plan

- **Git:**
  - The `main` branch is kept stable and is the source of the published documents.
  - Work is done on a separate branch for each issue, named after the issue number (e.g. `feature/18`).
  - Changes are merged into `main` through pull requests, which must be reviewed and approved by at least one other team member and pass CI before merging. Each pull request links the issue it resolves (e.g. `Closes #18`) so the issue closes automatically when it is merged.
  - After a pull request is merged, its branch is deleted on GitHub and locally. Branches that have had no activity for over two weeks are reviewed at the next team meeting and either finished or deleted, so that the list of branches only shows active work.
- **Tags and releases:**
  - When a deliverable is submitted, the commit on `main` that was submitted is marked with an annotated Git tag named after the deliverable and revision (e.g. `DevPlan-Rev0`, `SRS-Rev0`, `Final-Rev1`). This records exactly what was submitted, and makes it easy to compare later revisions against it.
  - Tags are created only after the deliverable's pull request has been merged and the quality assurance check is complete.
  - Once working software exists, each demonstration (the proof of concept, Revision 0, and Revision 1) is also published as a GitHub Release from its tag, with a short summary of the features included.
- **Issues:**
  - Every task, bug, and meeting is tracked as a GitHub issue.
  - Issue templates are used for recurring issue types: team meetings, supervisor meetings, TA meetings, lectures, and peer reviews. These templates automatically apply the `meeting` or `review` label.

  - Other issues, such as development tasks and bugs, are created as blank issues. These are classified with labels that describe the type of work, such as `documentation`, `enhancement`, and `bug`.
  - Each issue is assigned to a team member and tracked on the project's Kanban board.

- **CI/CD:**
  - A GitHub Actions workflow compiles the LaTeX documents on every pull request to `main`, so documents that fail to build are caught before merging.
  - On every push to `main`, the same workflow rebuilds only the documents affected by the change, commits the generated PDFs to the `pdfs/` folder, and deploys them to GitHub Pages for review.
  - Once source code is added, the CI pipeline will be extended to build the project and run the unit tests on every pull request.

8 Project Decomposition and Scheduling

The team will use GitHub Projects to organize and track the development of the project. A Kanban board has been created with the following stages: *Backlog*, *Ready*, *In Progress*, *In Review*, and *Done*. Tasks will move through these stages as work is planned, implemented, reviewed, and completed.

The project is decomposed into smaller GitHub Issues representing development tasks, research activities, meetings, and course deliverables. Labels are used to identify the type of work, such as research, development plan tasks, problem statement tasks, and meetings. This allows the team to assign work to individual members and monitor the progress of each task.

The GitHub Project can be found at:

https://github.com/orgs/SoftwareMandems/projects/1

The overall project schedule will follow the major capstone milestones and deadlines provided in the course outline. In addition to course deliverables, technical work will be scheduled in stages. The early stages will focus on setting up the Unity and VR environment and developing a basic FPV drone flight model. Once the core simulation is working, development will move toward controller support, training environments, scoring and feedback systems, and then the stretch goals: multiplayer functionality, AI-based flight analysis, user accounts, and database storage of flight records.

Tasks that are not yet ready to be worked on will remain in the backlog, while tasks that are prepared for development will move into the ready state. Work currently being completed will be tracked as in progress, and completed work will be reviewed before being moved to done. This workflow will allow the team to continuously track progress and adjust the schedule as priorities change.

9 Proof of Concept Demonstration Plan

The main appeal for this project and what should be prioritized above all else is the VR integration and controller interface support. Without it, the entire idea falls flat and loses its biggest charm. To prevent that from happening, the proof of concept demonstration will generally focus on setting up a basic Unity project with VR support along with a simple but functional drone flight model and an off-the-shelf USB controller. By following this plan, the team will be able to demonstrate the feasibility of the overall concept while establishing a foundation for the remainder of the project's development.

9.1 Potential Risks and Mitigation Strategies

The following is a list of potential risks that can impact the success of FPVR:

- **VR Incompatibility:** VR hardware (Meta Quest 3) may face compatibility issues with Unity or VR SDK. Begin VR integration early using Unity's OpenXR Plugin and XR Interaction Toolkit. Test the Meta Quest 3 with basic scenes and build off from there. If for whatever reason this is not possible, one fallback could be to create a non-VR desktop mode for users to interact with the simulation.
- **RF Integration Issues:** Interfacing open-source radio communication protocols with a custom RF module to connect with drone controllers may present challenges especially in integration. It is still unknown if such an idea is viable and more research is required to determine if it could work. Test early using recommended hardware, read through necessary documentation, and create simple prototypes. If a radio link is not feasible, the project will use a supported USB or virtual controller as an alternative solution.
- **Unrealistic Flight Physics:** The drone in simulation may not be as accurate as expected, leading to unrealistic flight and control behavior. Develop the flight model incrementally, beginning with basic movement and control before adding more advanced features such as PID feedback and telemetry. The simulation may be compared against real drone flight data, documented specifications, and feedback from experienced drone

pilots where possible. Physics parameters will be configurable so that the model can be adjusted as testing reveals inaccuracies.

- **Latency and Overall Performance:** The simulation will be built under dozens of hidden abstraction layers which can introduce unwanted latency and tank performance. Establish requirements for frame rate, input latency, and simulation response time early in development. Use Unity Profiler, Unity Memory Profiler, and Frame Debugger to identify bottlenecks. The team will attempt to focus on optimizing as much as possible, reducing unnecessary processing, rendering complexity, and network traffic. If all goes well and secondary features come into play, Multiplayer and AI-assisted tooling will be designed so that they do not interfere with the real-time flight controls.

10 Expected Technology

The project will use an open source game engine like Unity as the primary game engine and C# as the primary programming language. C# will be used to implement the drone simulation, flight controls, physics, user interfaces, VR interactions, multiplayer functionality, and AI-based flight analysis. Unity and C# were selected because they provide established tools for VR development, physics simulation, and cross-platform deployment. However, Unity's abstractions may limit access to lower-level hardware functionality and may introduce performance overhead for computationally intensive components. C may therefore be investigated for lower-level components such as drivers if direct hardware access or performance requirements make it necessary.

Unity's built-in physics, input, and rendering systems will be used where appropriate to support the drone simulation. These systems reduce the need to implement common functionality from scratch, but their general-purpose implementations may not provide the level of control or performance required for realistic FPV flight. Unity's VR and XR packages such as OpenXR Plugin and XR Interaction Toolkit will be investigated for integration with the project's VR hardware, with compatibility and input latency evaluated during development. Additional Unity packages and libraries such as Unity Multiplayer SDK, Unity Transport and Unity Netcode may be used for multiplayer networking, data collection, visualization, and other functionality required by the project. These packages will be evaluated based on their compatibility, performance, documentation, and ability to meet the project's requirements, with custom implementations considered where existing functionality is insufficient.

For the AI-based flight analysis component, pre trained models are expected to be leveraged rather than developing a model from scratch. Open Source models like Qwen2-VL may be investigated to provide feedback on areas such as control

efficiency and flight performance. However, general-purpose models may not reliably interpret the structured controller inputs and flight telemetry required by the project and may produce inconsistent feedback. Existing models and libraries will therefore be tested against representative flight data to determine whether they satisfy the project's accuracy, performance, and hardware requirements before considering fine-tuning, adapting an existing model, or developing a custom model.

The project can use a standard linter such as Microsoft C# linter and formatter to maintain consistent coding conventions and identify potential issues during development. The linter will provide automated checks for common coding problems, although static analysis cannot detect all runtime or simulation-specific issues. Unit testing will therefore be performed using an appropriate C# testing framework, such as NUnit or Unity's Test Framework. Code coverage tools will also be investigated to measure the extent to which automated tests exercise the implemented functionality and identify areas requiring additional tests.

Continuous Integration (CI) will be implemented using GitHub Actions where practical. Automated builds, tests, and code-quality checks will be run when changes are pushed to the repository or submitted through pull requests. GitHub Actions was selected because it integrates with the team's existing GitHub workflow and can automate repetitive verification tasks. However, Unity projects can require significant build dependencies and execution time, so the CI workflow will be limited to checks that can be reliably automated within the available resources. This will help identify integration and testing issues before changes are merged into the main branch.

Git, GitHub, and GitHub Projects will be used for version control, collaboration, issue tracking, and project management. GitHub Issues will be used to track development tasks and defects, while GitHub Projects will be used to organize and monitor the project's overall progress. These tools provide an integrated workflow for coordinating development, while working within the github ecosystem.

The team will use an open source IDE like Visual Studio Code as the primary IDE for development, debugging, and integration with Unity. Visual Studio Code provides a lightweight development environment and supports the project's required languages and tools, although its Unity and C# functionality may not provide the same level of integrated tooling as a full-featured IDE but it gives familiarity and flexibility if changes are to be made in the future. Code coverage can be measured using a separate Unity's Code Coverage package to determine the percentage of C# code executed by the project's automated tests.

The team will use a free performance profiler such as the Unity Profiler to

measure CPU, GPU, memory, physics, and rendering performance throughout development. The Unity Memory Profiler and Frame Debugger will also be investigated to identify memory usage issues and rendering bottlenecks, particularly for maintaining the performance requirements of the VR simulation. These tools provide detailed information about Unity's runtime behaviour, but profiling results may not fully represent performance on the final VR hardware. Performance testing will therefore also be performed on the target hardware to validate that optimizations identified through profiling translate into acceptable real-world performance.

Although existing Unity systems, libraries, and frameworks will be used where appropriate, major project-specific components such as the drone flight model, controller behaviour, flight evaluation system, and integration between the simulation, VR controls, multiplayer functionality, and AI analysis will be implemented through custom code. Existing tools will be preferred when they provide reliable and maintainable functionality, while custom implementations will be considered when their limitations prevent them from meeting the project's requirements.

11 Use of AI and LLMs

The team will use Claude Code and ChatGPT as development and documentation assistance tools throughout the project. These tools may be used to assist with generating code, explaining unfamiliar concepts, debugging, developing test cases, reviewing code, and drafting or improving project documentation.

LLM-generated code and documentation will not be treated as the standard. All generated code will be reviewed and tested by team members before being incorporated into the project. The team will verify functionality through unit tests, integration testing, manual testing, and performance testing where appropriate. Documentation generated with the assistance of LLMs will also be reviewed by team members against project requirements and rubrics to ensure that it is accurate.

12 Coding Standard

The project will use the **Microsoft C# Coding Conventions** as the coding standard for all C# source code. This will provide consistent conventions for naming, formatting, organization, and documentation throughout the project.

PascalCase will be used for classes, methods, and public properties, and camel-

Case for local variables and parameters. Methods and classes will have descriptive names, and code will be organized into modular classes with each class having a clear responsibility.

The project will use consistent indentation and brace formatting, avoid unnecessary code duplication, and include comments where they help explain non-obvious implementation details. Code will be formatted consistently using the development environment's formatting tools. Pull requests will be reviewed by another team member before being merged into the main branch to ensure the coding standard is followed.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- ChatGPT 5.6 Sol, OpenAI
- Claude Opus 4.8, Anthropic

13.2 How and Where Used

ChatGPT 5.6 Sol and Claude Opus 4.8 were used as support tools for selected sections of this document. They were used for grammar and wording review, brainstorming and research help, and checking consistency between sections. Final decisions about project requirements, technologies, responsibilities, workflows, and wording were made by the project team.

AI assistance was used in the following sections:

- **Team Meeting Plan — ChatGPT 5.6 Sol:** ChatGPT reviewed the wording and organization of meeting procedures and checked for conflicts with the Team Communication Plan and Team Member Roles. The meeting process was decided by the team.
- **Team Communication Plan — Claude Opus 4.8:** Claude reviewed the section for clarity and checked that the uses of Discord, GitHub Issues, pull request comments, and external communication were clearly explained and consistent with related sections.
- **Team Member Roles — ChatGPT 5.6 Sol:** ChatGPT reviewed project and technical role descriptions and checked that responsibilities matched related sections. All roles were assigned by the team before the AI review.
- **Workflow Plan — Claude Opus 4.8:** Claude reviewed the Git, GitHub, issue tracking, pull request, release, and CI/CD workflows for clarity and consistency. The workflow itself was designed by the team.
- **Project Decomposition and Scheduling — ChatGPT 5.6 Sol:** ChatGPT reviewed the explanation of GitHub Issues and the Kanban board and helped make the workflow states easier to understand. Scheduling, milestones, assignments, and priorities were decided by the team.
- **Expected Technology — ChatGPT 5.6 Sol and Claude Opus 4.8:** Both tools were used for brainstorming, research help, and wording review. ChatGPT helped identify possible technologies, libraries, testing tools, profiling tools, and approaches related to Unity, C#, VR/XR, multiplayer, testing, CI, performance, and AI-based flight analysis.

Claude provided an additional review and helped identify claims that were too certain, missing trade-offs, or technologies that should be described as options to investigate.

AI-generated research was treated only as a starting point. Important technical claims were checked against suitable primary sources, such as official documentation.

- **Coding Standard — ChatGPT 5.6 Sol:** ChatGPT reviewed the section for clarity and organization. The Microsoft C# Coding Conventions and project-specific practices were chosen by the team and checked against the original standard.

AI was also used to compare related sections for differences in terminology, responsibilities, communication methods, review procedures, and project-management processes. Any possible conflict was checked against the team's agreed-upon workflow before changes were made.

AI was also used to help summarize and shorten this Generative AI Use Disclosure while keeping the required information.

Neither ChatGPT nor Claude was treated as an official source for project requirements, technical facts, team responsibilities, or project-management decisions.

13.3 Verification of AI-Generated Content

All AI-assisted material was manually reviewed before being added to the official document. Every AI-assisted change included in the final version was reviewed by at least two team members.

Grammar and wording changes were checked to ensure they improved readability without changing technical meaning or team decisions. For team organization sections, reviewers checked that responsibilities, meeting procedures, communication methods, GitHub processes, and review procedures remained consistent.

For the Expected Technology section, AI suggestions were treated as possible options rather than confirmed facts. Relevant technical claims were checked against primary or official sources, and technologies that had not been fully researched were described as options to investigate.

Agreement between ChatGPT and Claude was not considered verification by itself. Human review was still required.

The team remains responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document. All AI-assisted material was re-

viewed by at least two team members and checked against other sources where needed.

Appendix — Reflection

Sankalpa — Reflection

Why is it important to create a development plan prior to starting the project?

Creating a development plan before starting the project is important because it gives the team a shared understanding of how the project will be organized and developed. Without a plan, it would be easy for team members to have different expectations about responsibilities, workflows, deadlines, and development practices. While working on the Project Decomposition and Scheduling section, I found that breaking the project into GitHub Issues and organizing them on a Kanban board made the overall project feel much more manageable. Instead of viewing the FPV drone simulator as one large project, we can approach it as smaller tasks that can be assigned, tracked, reviewed, and completed individually. I also think the plan will be useful later in the year because we now have an initial structure that we can adjust as our priorities and understanding of the project change.

In your opinion, what are the advantages and disadvantages of using CI/CD?

One of the biggest advantages of CI/CD is that it can catch problems early. Automated builds, tests, and other checks can run whenever changes are pushed or a pull request is created, which reduces the chance of broken or incompatible code being merged into the main branch. This is especially useful for a team project because multiple people will be developing different parts of the system at the same time. It also encourages the team to integrate changes frequently instead of waiting until large amounts of code have been written separately.

A disadvantage is that CI/CD requires additional setup and maintenance. For our project, which involves Unity and VR, configuring automated builds and tests may be more complicated than it would be for a simpler software project. A CI pipeline can also become frustrating if it is poorly configured and fails for reasons unrelated to the actual code. Because of this, I think CI/CD is valuable for our project, but we should automate checks where they provide a clear benefit rather than adding automation simply for the sake of having it.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group did not have any major disagreements while completing this deliverable. Most discussions were about how we wanted to organize the project and how much detail we should include at this early stage. For example, while setting up the project decomposition and scheduling, we had to decide how tasks should be represented and tracked through GitHub Issues and the Kanban board. We resolved these smaller differences through discussion and by choosing approaches that were simple enough to use now but could still be expanded later as the project becomes more complex. Overall, responsibilities were divided between team members, and we reviewed the combined work to make sure the different sections were consistent with the direction of the project.

Arjun - Reflection

Why is it important to create a development plan prior to starting the project?

A development plan forces the team to agree on how the project will run before any real work depends on those decisions. When I wrote the workflow plan, I had to decide how branches are named, when they are deleted, how pull requests are reviewed, and how submitted deliverables are tagged. These are easy to agree on now, but would be much harder to change after five people have each developed their own habits. Writing the team roles and meeting plan had a similar effect: naming a team lead, a team liaison, and a rotating quality assurance role means that responsibilities are clear from the start, instead of being assumed. The plan also acts as a reference point. When questions come up later in the year, such as who contacts the supervisor or how a decision should be recorded, the team can check the plan rather than debate the question again.

In your opinion, what are the advantages and disadvantages of using CI/CD?

The main advantage of CI/CD is that checks run automatically, so the team does not have to rely on each member remembering to run them. Our repository already compiles the LaTeX documents on every pull request, which means a document that fails to build is caught before it is merged into `main`. The same workflow also publishes the generated PDFs, so reviewers and the TA always see the latest version without anyone building and uploading them by hand. Once our Unity code is added, running the build and unit tests on every pull request should give the same protection for the software.

The disadvantage is that the pipeline itself becomes something the team has to

understand and maintain. While working on this deliverable, I found that the workflow only produces a viewable PDF after changes are merged into `main`, which was not obvious at first. Workflows can also fail for reasons unrelated to our work, such as package installation problems, and each run takes time before a pull request can be merged. For a Unity and VR project, automating builds and tests will take extra setup, so the team should add automation gradually where it clearly saves effort.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

The team did not have any serious disagreements, but there were a few decisions that needed discussion. The main one was how often the team should meet. The first draft of the meeting plan had two meetings a week, one in person and one on Discord. After discussing everyone's schedules, the team agreed that one in-person meeting on Thursday afternoons was more realistic, with Discord used as a backup when the team cannot meet in person. There was also some discussion about how specific the team roles should be, since some technical areas, such as the back-end systems, are not fully defined yet. The team resolved this by assigning each member a technical area now and stating that roles are expected to change over the year. These decisions were made through discussion in meetings and on Discord, and the final wording was checked through pull request reviews.

Appendix — Team Charter

External Goals

The team's main external goal is to produce a polished capstone project that we are proud to demonstrate at the Capstone EXPO and can discuss in future academic and professional opportunities. We aim to perform well in the course while also gaining practical experience with technologies and development practices that are relevant outside of the classroom. Each team member should finish the project with meaningful technical contributions that they can discuss in interviews and include in their portfolio.

Attendance

Expectations

Team members are expected to attend at least 90% of scheduled team and supervisor meetings and arrive on time. If a member knows that they will be late or unable to attend, they should notify the team through Discord as early as possible.

Members who miss a meeting are responsible for reviewing the meeting notes and completing any action items assigned to them. Attendance will be reviewed if a member repeatedly misses meetings or arrives late without communicating with the team.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, emergencies, unavoidable academic conflicts, work obligations that cannot reasonably be changed, or other circumstances outside of a member's control.

Whenever possible, the team should be notified in advance. Forgetting about a meeting, repeatedly failing to manage time, or missing a deadline without communicating with the team are not considered acceptable excuses.

In Case of Emergency

If a team member experiences an emergency that prevents them from attending a meeting or completing assigned work, they should notify the team through Discord as soon as reasonably possible.

If an important task cannot be completed before its deadline, the member should

provide the team with the current state of their work when possible. The team will then redistribute urgent work among available members and adjust responsibilities or deadlines where possible.

Accountability and Teamwork

Quality

Team members are expected to come to meetings prepared and complete assigned work to a standard suitable for inclusion in the project. Members should review and test their own work before requesting review from another team member.

Course deliverables should be checked against the relevant rubric and project requirements before submission. Pull requests should contain work that is understandable and ready for review. At least one other team member should review a pull request before it is merged.

Attitude

Team members are expected to communicate respectfully and be open to the ideas and feedback of others. Members should be comfortable proposing ideas, asking questions, disagreeing with decisions, and providing constructive criticism.

Everyone should make a reasonable effort to help other members when they encounter blockers. Disagreements should focus on the project rather than individuals, and criticism should be constructive.

Stay on Track

GitHub Issues and the GitHub Project Kanban board will be used to track assigned work, deadlines, and progress. Each task should have an assigned team member and be moved through the board as work progresses. Progress and blockers will also be discussed during regular team meetings.

Trigger items that may require team discussion include repeatedly missing meetings, failing to communicate for more than 48 hours on weekdays, repeatedly missing agreed-upon deadlines without notice, or consistently failing to complete assigned work.

When one of these issues occurs, the team will first discuss it directly with the member and determine whether there is a blocker or other issue that can be resolved. If the behaviour continues, responsibilities may be reassigned and the

issue will be discussed by the full team. Continued problems may be brought to the TA, supervisor, or instructor if necessary.

Members who complete their work early or make additional contributions are encouraged to help with outstanding development, testing, reviews, or documentation. Strong contributions will also be recognized during meetings and considered when dividing future work so that responsibilities remain fair. Contributors that continously fail to meet expectations will have to buy lunch for everyone on the team, in the next meeting.

Team Building

The team will build cohesion through regular in-person meetings and collaborative working sessions. Discord will also be used for informal communication so that interactions are not limited strictly to assigned project tasks.

The team may occasionally organize informal activities such as getting food together, playing poker, or working together outside of scheduled meetings. The goal is to maintain a comfortable team environment where members can communicate openly and enjoy working together throughout the project.

Decision Making

The team will attempt to make decisions through consensus whenever possible. Each member should have an opportunity to give their opinion, particularly when a decision directly affects the technical area they are leading.

For technical disagreements, the team will consider project requirements, feasibility, implementation effort, maintainability, performance, and evidence from research or testing. When practical, prototypes or experiments may be used to compare alternatives.

If consensus cannot be reached within a reasonable amount of time, the team will use a majority vote so that the disagreement does not block progress. With five team members, a decision requires at least three votes to pass. For significant decisions where the team remains uncertain, the team may consult the supervisor, TA, or instructor. Important decisions will be recorded in the relevant GitHub Issue.